🐳 Docker Full Stack Lab — Guida Pratica

React + Node.js + PostgreSQL + Nginx

ITS Pordenone | Virtualizzazione e Containerizzazione

🎯 Obiettivo

Costruire e avviare un'applicazione **Task Manager** full stack completamente dockerizzata:

Servizio	Tecnologia	Ruolo	Porta interna
postgres	PostgreSQL 16	Database relazionale	5432
node	Node.js 20 + Express	REST API Backend	4000
react	React + Vite (build), Nginx (serve)	Frontend SPA	3000
nginx	Nginx	Reverse Proxy (unica porta host)	**80 → host**

Flusso: Browser → Nginx:80 → React (/) oppure Node (/api) → PostgreSQL

⏱️ Piano Lab (4 ore)

Tempo	Fase	Attività
0:00–0:20	Setup	Struttura cartelle, verifica Docker
0:20–0:50	PostgreSQL	init.sql, volume, healthcheck
0:50–1:30	Node.js Backend	Express API, Dockerfile, test curl
1:30–2:15	React Frontend	App.jsx, Dockerfile multi-stage
2:15–2:40	Nginx	Reverse proxy config
2:40–3:15	Docker Compose	docker-compose.yml completo, avvio
3:15–4:00	Test & Debug	Verifica end-to-end, troubleshooting

📁 Struttura Progetto Finale

```
fullstack-app/
├── docker-compose.yml
├── postgres/
│   └── init.sql
├── backend/
│   ├── Dockerfile
│   ├── package.json
```

```
│   └── server.js
├── frontend/
│   ├── Dockerfile
│   ├── nginx-react.conf
│   ├── package.json
│   ├── vite.config.js
│   └── src/
│       ├── main.jsx
│       └── App.jsx
└── nginx/
    └── nginx.conf
```

🔧 Fase 1 — Setup (20 min)

Verifica Docker

```
docker --version
docker compose version
docker run --rm hello-world
```

Crea la struttura cartelle

```
mkdir ~/fullstack-app && cd ~/fullstack-app
mkdir -p postgres backend frontend/src nginx
```

🗄️ Fase 2 — PostgreSQL (30 min)

2.1 — Script di inizializzazione

```
cat > postgres/init.sql << 'EOF'
-- Tabella principale tasks
CREATE TABLE IF NOT EXISTS tasks (
  id         SERIAL PRIMARY KEY,
  titolo     VARCHAR(200) NOT NULL,
  completato BOOLEAN      DEFAULT FALSE,
  creato_il  TIMESTAMP    DEFAULT NOW()
);

-- Dati di esempio per non avere la lista vuota al primo avvio
INSERT INTO tasks (titolo) VALUES
  ('Completare il lab Docker'),
  ('Studiare Docker Compose'),
  ('Buildare il frontend React');
EOF
```

2.2 — Test isolato PostgreSQL

Prima di integrare tutto, testa Postgres da solo:

```
# Avvia Postgres in standalone
docker run -d --name test-pg \
  -e POSTGRES_DB=taskdb \
  -e POSTGRES_USER=taskuser \
  -e POSTGRES_PASSWORD=taskpass \
  -v $(pwd)/postgres/init.sql:/docker-entrypoint-initdb.d/init.sql \
  postgres:16-alpine

# Aspetta l'avvio (circa 10 secondi)
docker logs -f test-pg
# Ctrl+C quando vedi "database system is ready to accept connections"

# Connettiti e verifica la tabella e i dati
docker exec -it test-pg psql -U taskuser -d taskdb
```

Dentro psql:

```
\dt                         -- elenca le tabelle
SELECT * FROM tasks;        -- vedi i dati inseriti
\q                          -- esci
```

```
# Cleanup test
docker stop test-pg && docker rm test-pg
```

□ Fase 3 — Node.js Backend (40 min)

3.1 — package.json

```
cat > backend/package.json << 'EOF'
{
  "name": "taskapi",
  "version": "1.0.0",
  "main": "server.js",
  "dependencies": {
    "express": "^4.18.0",
    "pg": "^8.11.0",
    "cors": "^2.8.5"
  }
```

```
}
EOF
```

3.2 — server.js (API REST completa)

```
cat > backend/server.js << 'EOF'
const express = require('express');
const { Pool } = require('pg');
const cors    = require('cors');

const app = express();
app.use(cors());
app.use(express.json());

// Pool connessione PostgreSQL
// I valori vengono da variabili d'ambiente (settate nel docker-compose.yml)
const pool = new Pool({
  host:     process.env.DB_HOST     || 'localhost',
  database: process.env.DB_NAME     || 'taskdb',
  user:     process.env.DB_USER     || 'taskuser',
  password: process.env.DB_PASS     || 'taskpass',
  port:     process.env.DB_PORT     || 5432,
});

// ── GET /api/tasks — lista tutte le task
app.get('/api/tasks', async (req, res) => {
  try {
    const { rows } = await pool.query(
      'SELECT * FROM tasks ORDER BY creato_il DESC'
    );
    res.json(rows);
  } catch (err) {
    console.error('GET /api/tasks error:', err.message);
    res.status(500).json({ error: err.message });
  }
});

// ── POST /api/tasks — crea nuova task
app.post('/api/tasks', async (req, res) => {
  const { titolo } = req.body;
  if (!titolo || !titolo.trim()) {
    return res.status(400).json({ error: 'titolo è obbligatorio' });
  }
  try {
    const { rows } = await pool.query(
      'INSERT INTO tasks(titolo) VALUES($1) RETURNING *',
      [titolo.trim()]
    );
    res.status(201).json(rows[0]);
  } catch (err) {
    console.error('POST /api/tasks error:', err.message);
```

```
    res.status(500).json({ error: err.message });
  }
});

// ── PATCH /api/tasks/:id/toggle — inverte completato
app.patch('/api/tasks/:id/toggle', async (req, res) => {
  try {
    const { rows } = await pool.query(
      'UPDATE tasks SET completato = NOT completato WHERE id=$1 RETURNING *',
      [req.params.id]
    );
    if (rows.length === 0) return res.status(404).json({ error: 'task non trovata'
});
    res.json(rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// ── DELETE /api/tasks/:id — elimina task
app.delete('/api/tasks/:id', async (req, res) => {
  try {
    const result = await pool.query('DELETE FROM tasks WHERE id=$1',
[req.params.id]);
    if (result.rowCount === 0) return res.status(404).json({ error: 'task non
trovata' });
    res.status(204).end();
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// ── GET /api/health — health check (usato da Nginx e monitoring)
app.get('/api/health', (req, res) => {
  res.json({ status: 'ok', timestamp: new Date().toISOString() });
});

const PORT = process.env.PORT || 4000;
app.listen(PORT, () => {
  console.log(`🚀 API server running on port ${PORT}`);
  console.log(`   DB: ${process.env.DB_HOST}:${process.env.DB_PORT || 5432}`);
});
EOF
```

3.3 — Dockerfile backend

```
cat > backend/Dockerfile << 'EOF'
FROM node:20-slim

WORKDIR /app
```

```
# Copia package.json prima per sfruttare la cache dei layer Docker
COPY package*.json ./
RUN npm install --production

# Poi copia il codice sorgente
COPY . .

# Variabili d'ambiente con valori di default
# Vengono sovrascritte dal docker-compose.yml
ENV NODE_ENV=production \
    PORT=4000 \
    DB_HOST=postgres \
    DB_PORT=5432 \
    DB_NAME=taskdb \
    DB_USER=taskuser \
    DB_PASS=taskpass

EXPOSE 4000

# Non eseguire come root
USER node

CMD ["node", "server.js"]
EOF
```

3.4 — Test backend isolato

```
cd backend && npm install && cd ..

# Build immagine backend
docker build -t taskapi:test ./backend

# Avvia Postgres per il test
docker network create test-net
docker run -d --name test-pg --network test-net \
  -e POSTGRES_DB=taskdb -e POSTGRES_USER=taskuser -e POSTGRES_PASSWORD=taskpass \
  -v $(pwd)/postgres/init.sql:/docker-entrypoint-initdb.d/init.sql \
  postgres:16-alpine

# Aspetta Postgres (~15s)
sleep 20

# Avvia il backend
docker run -d --name test-api --network test-net \
  -p 4000:4000 \
  -e DB_HOST=test-pg \
  taskapi:test

# Test delle API
curl http://localhost:4000/api/health
# → {"status":"ok","timestamp":"..."}
```

```
curl http://localhost:4000/api/tasks
# → [{"id":1,"titolo":"Completare il lab Docker",...}, ...]

curl -X POST http://localhost:4000/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"titolo":"Task creata via curl"}'
# → {"id":4,"titolo":"Task creata via curl","completato":false,...}

curl -X PATCH http://localhost:4000/api/tasks/1/toggle
# → {"id":1,"titolo":"...","completato":true,...}

curl -X DELETE http://localhost:4000/api/tasks/4
# → 204 No Content

# Cleanup test
docker stop test-api test-pg && docker rm test-api test-pg
docker network rm test-net
```

⚛ Fase 4 — React Frontend (45 min)

4.1 — Inizializza progetto Vite

```
cd frontend

# Installa Node.js sull'host se non presente (solo per sviluppo)
# In alternativa, usa l'immagine Docker per la build
npm create vite@latest . -- --template react
# Premi Invio per confermare nella directory corrente
# Seleziona: React → JavaScript

npm install
cd ..
```

4.2 — vite.config.js

```
cat > frontend/vite.config.js << 'EOF'
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    port: 3000,
    host: true,    // ascolta su 0.0.0.0 (necessario per Docker)
  },
  build: {
    outDir: 'dist',
```

```
  }
})
EOF
```

4.3 — App.jsx (Task Manager completo)

```
cat > frontend/src/App.jsx << 'APPEOF'
import { useState, useEffect } from "react";

// /api viene proxato da Nginx verso il backend Node.js
// In questo modo React non ha bisogno di conoscere l'indirizzo del backend!
const API = "/api";

export default function App() {
  const [tasks, setTasks]   = useState([]);
  const [input, setInput]   = useState("");
  const [loading, setLoading] = useState(true);
  const [error, setError]   = useState(null);

  // Carica tasks dal backend
  const loadTasks = async () => {
    try {
      const res = await fetch(`${API}/tasks`);
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      const data = await res.json();
      setTasks(data);
      setError(null);
    } catch (err) {
      setError(`Errore caricamento: ${err.message}`);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => { loadTasks(); }, []);

  // Aggiunge nuova task
  const addTask = async () => {
    const titolo = input.trim();
    if (!titolo) return;
    try {
      const res = await fetch(`${API}/tasks`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ titolo }),
      });
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      setInput("");
      loadTasks();
    } catch (err) {
      setError(`Errore aggiunta: ${err.message}`);
```

```
    }
  };

  // Toggle completato
  const toggleTask = async (id) => {
    try {
      await fetch(`${API}/tasks/${id}/toggle`, { method: "PATCH" });
      loadTasks();
    } catch (err) {
      setError(`Errore toggle: ${err.message}`);
    }
  };

  // Elimina task
  const deleteTask = async (id) => {
    try {
      await fetch(`${API}/tasks/${id}`, { method: "DELETE" });
      loadTasks();
    } catch (err) {
      setError(`Errore eliminazione: ${err.message}`);
    }
  };

  const completate = tasks.filter(t => t.completato).length;

  return (
    <div style={styles.container}>
      <div style={styles.card}>
        {/* Header */}
        <div style={styles.header}>
          <h1 style={styles.title}>📋 Task Manager</h1>
          <p style={styles.subtitle}>
            React + Node.js + PostgreSQL + Nginx
          </p>
        </div>

        {/* Statistiche */}
        <div style={styles.stats}>
          <span>📋 Totale: <strong>{tasks.length}</strong></span>
          <span>✅ Completate: <strong>{completate}</strong></span>
          <span>⏳ In corso: <strong>{tasks.length - completate}</strong></span>
        </div>

        {/* Input nuova task */}
        <div style={styles.inputRow}>
          <input
            style={styles.input}
            value={input}
            onChange={e => setInput(e.target.value)}
            onKeyDown={e => e.key === "Enter" && addTask()}
            placeholder="Scrivi una nuova task e premi Invio..."
          />
          <button style={styles.btnAdd} onClick={addTask}>
            Aggiungi
```

```
            </button>
          </div>

          {/* Errori */}
          {error && (
            <div style={styles.error}>⚠ {error}</div>
          )}

          {/* Lista tasks */}
          {loading ? (
            <p style={{ textAlign: "center", color: "#666" }}>
              Caricamento...
            </p>
          ) : tasks.length === 0 ? (
            <p style={{ textAlign: "center", color: "#999" }}>
              Nessuna task. Aggiungine una!
            </p>
          ) : (
            <ul style={styles.list}>
              {tasks.map(task => (
                <li key={task.id} style={{
                  ...styles.taskItem,
                  background: task.completato ? "#f0fdf4" : "#fff",
                  borderLeft: `4px solid ${task.completato ? "#22c55e" :
"#3b82f6"}`,
                }}>
                  <input
                    type="checkbox"
                    checked={task.completato}
                    onChange={() => toggleTask(task.id)}
                    style={{ width: 18, height: 18, cursor: "pointer" }}
                  />
                  <span style={{
                    flex: 1,
                    textDecoration: task.completato ? "line-through" : "none",
                    color: task.completato ? "#9ca3af" : "#111",
                    fontSize: 15,
                  }}>
                    {task.titolo}
                  </span>
                  <span style={styles.date}>
                    {new Date(task.creato_il).toLocaleDateString("it-IT")}
                  </span>
                  <button
                    style={styles.btnDelete}
                    onClick={() => deleteTask(task.id)}
                  >
                    🗑
                  </button>
                </li>
              ))}
            </ul>
          )}
```

```
          {/* Footer */}
          <div style={styles.footer}>
            Stack: React → Nginx → Node.js → PostgreSQL | Docker Compose
          </div>
        </div>
      </div>
    );
  }

  const styles = {
    container: {
      minHeight: "100vh",
      background: "linear-gradient(135deg, #667eea 0%, #764ba2 100%)",
      display: "flex",
      alignItems: "center",
      justifyContent: "center",
      padding: "20px",
      fontFamily: "-apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif",
    },
    card: {
      background: "#f8fafc",
      borderRadius: 16,
      boxShadow: "0 20px 60px rgba(0,0,0,0.3)",
      width: "100%",
      maxWidth: 600,
      overflow: "hidden",
    },
    header: {
      background: "linear-gradient(90deg, #1e40af, #7c3aed)",
      padding: "24px 28px 20px",
      color: "white",
    },
    title: { margin: 0, fontSize: 26, fontWeight: 700 },
    subtitle: { margin: "6px 0 0", opacity: 0.85, fontSize: 13 },
    stats: {
      display: "flex",
      gap: 20,
      padding: "14px 28px",
      background: "#e0e7ff",
      fontSize: 14,
      color: "#374151",
    },
    inputRow: {
      display: "flex",
      gap: 10,
      padding: "18px 28px",
      borderBottom: "1px solid #e5e7eb",
    },
    input: {
      flex: 1,
      padding: "10px 14px",
      border: "1.5px solid #d1d5db",
      borderRadius: 8,
      fontSize: 15,
```

```
      outline: "none",
    },
    btnAdd: {
      padding: "10px 20px",
      background: "#3b82f6",
      color: "white",
      border: "none",
      borderRadius: 8,
      cursor: "pointer",
      fontWeight: 600,
      fontSize: 14,
    },
    error: {
      margin: "0 28px 14px",
      padding: "10px 14px",
      background: "#fef2f2",
      border: "1px solid #fca5a5",
      borderRadius: 8,
      color: "#dc2626",
      fontSize: 13,
    },
    list: {
      listStyle: "none",
      margin: 0,
      padding: "8px 28px 16px",
      display: "flex",
      flexDirection: "column",
      gap: 8,
    },
    taskItem: {
      display: "flex",
      alignItems: "center",
      gap: 12,
      padding: "12px 14px",
      borderRadius: 8,
      border: "1px solid #e5e7eb",
      transition: "all 0.15s",
    },
    date: { fontSize: 11, color: "#9ca3af" },
    btnDelete: {
      background: "none",
      border: "none",
      cursor: "pointer",
      fontSize: 16,
      padding: "2px 4px",
      borderRadius: 4,
      opacity: 0.6,
    },
    footer: {
      padding: "12px 28px",
      background: "#1e293b",
      color: "#64748b",
      fontSize: 11,
      textAlign: "center",
```

```
  },
};
APPEOF
```

4.4 — main.jsx

```
cat > frontend/src/main.jsx << 'EOF'
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
EOF
```

4.5 — Nginx config per React SPA (gestisce il routing client-side)

```
cat > frontend/nginx-react.conf << 'EOF'
server {
    listen 3000;
    root /usr/share/nginx/html;
    index index.html;

    # Gestione routing SPA: tutte le route tornano a index.html
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Cache assets statici
    location ~* \.(js|css|png|jpg|svg|ico)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}
EOF
```

4.6 — Dockerfile frontend (multi-stage)

```
cat > frontend/Dockerfile << 'EOF'
# ── Stage 1: Build React con Vite ──────────────────────────────
FROM node:20-slim AS builder

WORKDIR /app
```

```
COPY package*.json ./
RUN npm install

COPY . .

# Produce la cartella /app/dist con i file statici ottimizzati
RUN npm run build

# ── Stage 2: Serve i file statici con Nginx ──────────────────────
FROM nginx:alpine AS runner

# Copia i file buildati dallo stage 1
COPY --from=builder /app/dist /usr/share/nginx/html

# Configurazione Nginx per SPA React
COPY nginx-react.conf /etc/nginx/conf.d/default.conf

EXPOSE 3000

CMD ["nginx", "-g", "daemon off;"]
EOF
```

> 💡 **Multi-stage build**: lo stage `builder` usa Node.js (~180MB) per compilare React. Lo stage finale usa solo Nginx Alpine (~25MB) per servire i file statici. L'immagine finale NON contiene Node.js — molto più leggera e sicura!

4.7 — Test build frontend

```
# Prova la build localmente prima di integrare
cd frontend
npm run build
# Deve creare la cartella dist/

# Build immagine Docker
cd ..
docker build -t taskfrontend:test ./frontend
docker images | grep taskfrontend
```

⊕ Fase 5 — Nginx Reverse Proxy (25 min)

```
cat > nginx/nginx.conf << 'EOF'
upstream frontend {
    server react:3000;     # "react" = nome del servizio in docker-compose.yml
}

upstream backend {
    server node:4000;      # "node" = nome del servizio in docker-compose.yml
```

```
}

server {
    listen 80;

    # ── Tutto il traffico generico → Frontend React ──────────────
    location / {
        proxy_pass         http://frontend;
        proxy_http_version 1.1;
        proxy_set_header   Host              $host;
        proxy_set_header   X-Real-IP         $remote_addr;
        proxy_set_header   X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header   X-Forwarded-Proto $scheme;
    }

    # ── Tutte le chiamate /api/* → Backend Node.js ───────────────
    location /api {
        proxy_pass         http://backend;
        proxy_http_version 1.1;
        proxy_set_header   Host              $host;
        proxy_set_header   X-Real-IP         $remote_addr;
        proxy_set_header   X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header   X-Forwarded-Proto $scheme;

        # Timeout più lungo per query DB
        proxy_read_timeout 30s;
    }

    # ── Health check pubblico ────────────────────────────────────
    location = /health {
        access_log off;
        return 200 "Stack OK\n";
        add_header Content-Type text/plain;
    }
}
EOF
```

🎼 Fase 6 — Docker Compose (35 min)

6.1 — docker-compose.yml completo

```
cat > docker-compose.yml << 'EOF'
version: '3.9'

services:

  # ── 1. Database PostgreSQL ──────────────────────────────────────
  postgres:
    image: postgres:16-alpine
    restart: unless-stopped
```

```
    environment:
      POSTGRES_DB:       taskdb
      POSTGRES_USER:     taskuser
      POSTGRES_PASSWORD: taskpass
    volumes:
      - pgdata:/var/lib/postgresql/data       # dati persistenti
      - ./postgres/init.sql:/docker-entrypoint-initdb.d/init.sql  # schema
iniziale
    networks:
      - fullstack_net
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U taskuser -d taskdb"]
      interval: 10s
      timeout:  5s
      retries:  5

  # ── 2. Backend Node.js ──────────────────────────────────────────
  node:
    build: ./backend
    restart: unless-stopped
    environment:
      DB_HOST: postgres       # ← nome servizio = hostname Docker DNS
      DB_PORT: 5432
      DB_NAME: taskdb
      DB_USER: taskuser
      DB_PASS: taskpass
      PORT:    4000
    networks:
      - fullstack_net
    depends_on:
      postgres:
        condition: service_healthy   # aspetta healthcheck OK

  # ── 3. Frontend React ───────────────────────────────────────────
  react:
    build: ./frontend
    restart: unless-stopped
    networks:
      - fullstack_net
    depends_on:
      - node

  # ── 4. Reverse Proxy Nginx ──────────────────────────────────────
  nginx:
    image: nginx:alpine
    restart: unless-stopped
    ports:
      - "80:80"             # ← UNICA porta esposta all'host
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    networks:
      - fullstack_net
    depends_on:
      - react
```

```
      - node

volumes:
  pgdata:                   # volume named per i dati Postgres

networks:
  fullstack_net:            # rete interna, DNS automatico tra servizi
    driver: bridge
EOF
```

6.2 — .dockerignore per il backend

```
cat > backend/.dockerignore << 'EOF'
node_modules/
.git/
*.log
.env
EOF
```

6.3 — .dockerignore per il frontend

```
cat > frontend/.dockerignore << 'EOF'
node_modules/
dist/
.git/
*.log
EOF
```

🚀 Fase 7 — Avvio e Test (45 min)

7.1 — Prima esecuzione

```
cd ~/fullstack-app

# Build di tutte le immagini e avvio
docker compose up -d --build

# La prima volta scarica le immagini base (~3-4 minuti)
# Le successive sono molto più veloci grazie alla cache
```

7.2 — Monitoraggio avvio

```
# Stato di tutti i servizi
docker compose ps

# Output atteso dopo ~60 secondi:
# NAME                    STATUS              PORTS
# fullstack-app-nginx-1   running             0.0.0.0:80->80/tcp
# fullstack-app-node-1    running
# fullstack-app-postgres-1 healthy
# fullstack-app-react-1   running

# Log in tempo reale (Ctrl+C per fermare)
docker compose logs -f

# Log per servizio specifico
docker compose logs -f postgres
docker compose logs -f node
```

7.3 — Test end-to-end

```
# 1. Health check Nginx
curl http://localhost/health
# → Stack OK

# 2. API health check (Nginx → Node)
curl http://localhost/api/health
# → {"status":"ok","timestamp":"..."}

# 3. Lista tasks (Nginx → Node → Postgres)
curl http://localhost/api/tasks
# → [{"id":1,"titolo":"Completare il lab Docker",...}, ...]

# 4. Crea nuova task
curl -X POST http://localhost/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"titolo":"Task creata durante il lab!"}'
# → {"id":4,"titolo":"Task creata durante il lab!","completato":false,...}

# 5. Toggle completato
curl -X PATCH http://localhost/api/tasks/1/toggle
# → {"id":1,...,"completato":true}

# 6. Elimina task
curl -X DELETE http://localhost/api/tasks/4
# → 204 No Content

# 7. Apri nel browser
# → http://localhost
```

7.4 — Verifica database

```
# Accedi direttamente a PostgreSQL
docker compose exec postgres psql -U taskuser -d taskdb

# Dentro psql:
\dt                                -- lista tabelle
SELECT * FROM tasks;               -- vedi tutte le task
SELECT COUNT(*) FROM tasks WHERE completato = TRUE;  -- task completate
\q                                 -- esci
```

7.5 — Test persistenza dati

```
# Ferma e rimuovi i container (ma NON i volumi)
docker compose down

# Verifica che il volume pgdata esiste ancora
docker volume ls | grep pgdata

# Riavvia lo stack
docker compose up -d

# I dati sono ancora lì!
curl http://localhost/api/tasks
```

🔬 Esercizi Aggiuntivi

Esercizio A — Aggiungere variabile ambiente dal file .env

```
# Crea file .env nella root del progetto
cat > .env << 'EOF'
POSTGRES_PASSWORD=nuovapassword
NODE_ENV=production
EOF
```

Modifica `docker-compose.yml` per usarlo:

```
postgres:
  environment:
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}  # dal file .env
```

```
docker compose up -d
```

Esercizio B — Scalare il backend

```
# 3 istanze del backend (richiede che nginx faccia load balancing)
docker compose up -d --scale node=3
docker compose ps

# nginx distribuirà le richieste tra le 3 istanze round-robin
for i in {1..9}; do curl -s http://localhost/api/health; echo; done
```

Esercizio C — Rebuild singolo servizio

```
# Modifica App.jsx (es. cambia il titolo)
# poi rebuild SOLO il frontend senza toccare DB e backend
docker compose build react
docker compose up -d --no-deps react

# Verifica il cambiamento
curl http://localhost
```

Esercizio D — Ispeziona la rete interna

```
# Vedi tutti i container sulla rete
docker network inspect fullstack-app_fullstack_net

# Test DNS interno (da dentro node, risolvi "postgres")
docker compose exec node sh -c "ping -c 2 postgres"
docker compose exec node sh -c "ping -c 2 react"
docker compose exec node sh -c "ping -c 2 nginx"
```

🚨 Troubleshooting

Il frontend non carica

```
docker compose logs react       # errori di build Vite?
docker compose logs nginx       # errori di proxy?
curl -v http://localhost/       # dettagli HTTP
```

Il backend non si connette al DB

```
docker compose logs node        # errori di connessione?
docker compose logs postgres    # il DB è pronto?
docker compose ps | grep postgres  # è "healthy"?
```

```
# Testa la connessione manualmente dal container node
docker compose exec node sh -c "nc -zv postgres 5432"
```

La tabella non esiste (init.sql non eseguito)

```
# init.sql viene eseguito SOLO al primo avvio con volume vuoto
# Se il volume esiste già, lo script viene ignorato!
docker compose down -v           # elimina il volume
docker compose up -d             # ricrea tutto da zero
```

Porta 80 già in uso

```
sudo lsof -i :80                # trova chi usa la porta
# Cambia la porta nel docker-compose.yml:
# ports:
#   - "8080:80"   # usa 8080 invece di 80
```

Rebuild non aggiorna l'app

```
docker compose build --no-cache react    # ignora la cache
docker compose up -d react
```

📋 Comandi Rapidi

```
# Avvio
docker compose up -d --build              # build + start
docker compose up -d                      # solo start (no build)

# Monitoraggio
docker compose ps                         # stato servizi
docker compose logs -f [SERVICE]          # log real-time
docker compose top                        # processi nei container

# Debug
docker compose exec postgres psql -U taskuser -d taskdb
docker compose exec node bash
docker compose exec nginx sh

# Rebuild
docker compose build [SERVICE]            # rebuilda immagine
docker compose up -d --no-deps [SERVICE]  # riavvia senza dipendenze
```

```
# Stop
docker compose stop                      # ferma senza rimuovere
docker compose down                      # ferma + rimuove
docker compose down -v                   # + elimina volumi (⚠ dati persi)
docker compose down --rmi local          # + elimina immagini buildate
```

ITS Pordenone — Docker Full Stack Lab | React + Node.js + PostgreSQL + Nginx